

Universidade do Minho



COMPUTAÇÃO GRÁFICA

Grupo 16

Fase 3



Eduarda Vieira, A104098

Leonor Cunha, A103997

Pedro Rebelo, A104091

Conteúdo

1.	Introdução	3
2.	Gerador – Superfícies de Bézier Bicúbicas	4
2.1	Formato do ficheiro .patch	4
2.2	Avaliação da superfície	4
2.3	Nível de tessellation – justificação	4
3.	Engine – Animações Temporais	5
3.1	Rotação animada.....	5
3.2	Translação animada – Curva Catmull-Rom.....	5
3.3	Alinhamento à tangente (align)	5
4.	Engine – VBOs com Índices (EBO)	6
4.1	Dois buffers na GPU	6
4.2	Upload e desenho.....	6
5.	Parser XML – Extensões da Fase 3.....	7
6.	Cena de Demonstração – Sistema Solar Dinâmico	7
6.1	Princípio geral de escalagem.....	7
7.	Cometa – Curva Catmull-Rom	10
7.1	Número de pontos de controlo	10
8.	Compilação e Execução	10
10.	Conclusão	11

1. Introdução

A Fase 3 do trabalho prático de Computação Gráfica estende o motor desenvolvido na Fase 2 com três contribuições principais: suporte a superfícies de Bézier bicúbicas no gerador de primitivas, animações baseadas em curvas Catmull-Rom e rotações temporizadas no engine, e migração do modo imediato para VBOs (Vertex Buffer Objects) com índices (EBO).

O gerador passou a aceitar ficheiros .patch como entrada e a produzir modelos com tesselação usando a formulação matricial de Bézier. O engine passou a ler atributos time e align nos elementos translate e rotate do XML, calculando posições e orientações em tempo real. A renderização foi reescrita para usar glDrawElements com dois buffers na GPU, preparando a estrutura para a Fase 4.

A cena de demonstração é um sistema solar dinâmico: todos os planetas orbitam animados por rotações temporizadas, e um cometa percorre uma trajetória elíptica definida por uma curva Catmull-Rom, alinhando-se automaticamente à tangente do percurso.

2. Gerador – Superfícies de Bézier Bicúbicas

Foi adicionado ao gerador o comando `bezier`, que recebe um ficheiro `.patch`, um nível de tessellation e o nome do ficheiro de saída. A superfície resultante é guardada no mesmo formato de triângulos usado pelas primitivas da Fase 1.

2.1 Formato do ficheiro `.patch`

O ficheiro `.patch` segue o formato standard usado na cadeira. A primeira linha contém o número de patches. Seguem-se linhas com 16 índices por patch (separados por vírgulas), onde cada índice referencia um ponto de controlo. Depois do bloco de patches vem o número de pontos de controlo, seguido de uma linha por ponto com as coordenadas `x, y, z` separadas por vírgulas.

```
./generator bezier teapot.patch 10 comet.3d
```

2.2 Avaliação da superfície

Cada patch é definido por uma grelha 4×4 de pontos de controlo. A fórmula de avaliação para um par $(u, v) \in [0,1]^2$ é:

$$P(u,v) = U \cdot M \cdot P \cdot M^T \cdot V^T$$

Onde $U = [u^3, u^2, u, 1]$, $V = [v^3, v^2, v, 1]$ e M é a matriz de Bézier cúbica. A implementação usa a abordagem de dois passos: para cada valor de u , avalia-se o Bézier cúbico em cada uma das 4 linhas do patch (obtendo 4 pontos intermédios), e depois avalia-se outro Bézier cúbico em v sobre esses 4 pontos.

2.3 Nível de tessellation – justificação

Para o cometa foi usado `tessLevel=10`, resultando em $2 \times 10^2 = 200$ triângulos por patch e 6400 triângulos no total (teapot tem 32 patches). Esta escolha resulta de um equilíbrio entre três fatores:

- Detalhe visual: abaixo de `tessLevel=5` as curvas do teapot ficam visivelmente poligonais, quebrando a ilusão de superfície suave.
- Desempenho: com `tessLevel=10` o modelo tem 6400 triângulos, valor confortável para renderização em tempo real sem impacto perceptível na frame rate.
- Coerência com a escala da cena: o cometa tem `scale(0.5, 0.5, 0.5)` na cena, pelo que um detalhe excessivo (`tessLevel=20+`) seria invisível ao utilizador.

Cada quadrilátero da grelha gera dois triângulos, totalizando $2 \times \text{tessLevel}^2$ triângulos por patch:

```
// Para cada célula (i,j) da grelha:
Vec3 p00 = bezierSurface(u0, v0, patch);
Vec3 p10 = bezierSurface(u1, v0, patch);
Vec3 p01 = bezierSurface(u0, v1, patch);
Vec3 p11 = bezierSurface(u1, v1, patch);
// Triângulo 1: p00, p10, p11
// Triângulo 2: p00, p11, p01
```

3. Engine – Animações Temporais

O engine foi estendido para suportar dois novos tipos de transformação animada: rotação com tempo e translação sobre curva Catmull-Rom.

3.1 Rotação animada

Quando o elemento rotate contém o atributo time em vez de angle, o objeto realiza uma rotação contínua de 360° no período especificado. O ângulo atual é calculado em cada frame com base no tempo decorrido desde o início da aplicação, obtido através de glutGet(GLUT_ELAPSED_TIME):

```
<rotate time="40" x="0" y="1" z="0" />

float elapsed = (float)glutGet(GLUT_ELAPSED_TIME) / 1000.0f;

float angle = fmod(elapsed / t.time * 360.0f, 360.0f);
glRotatef(angle, t.x, t.y, t.z);
```

O fmod garante que o ângulo faz um ciclo corretamente sem acumulação de erros de vírgula flutuante ao longo do tempo.

3.2 Translação animada – Curva Catmull-Rom

Quando o elemento translate contém o atributo time e pontos filhos, o objeto percorre uma curva Catmull-Rom definida por esses pontos.

```
<translate time="30" align="True">
  <point x="0" y="5" z="50" />
  ...
</translate>
```

A curva Catmull-Rom é definida pela equação matricial $P(t) = T \cdot M_CR \cdot P$, onde $T = [t^3, t^2, t, 1]$ e M_CR é a matriz de Catmull-Rom. Com N pontos, existem $N-3$ segmentos. Para um t global, determina-se o segmento correspondente e o t local dentro desse segmento, e avalia-se a equação com os 4 pontos do segmento.

A derivada (tangente) é calculada em simultâneo usando $dT = [3t^2, 2t, 1, 0]$, necessária para o alinhamento.

3.3 Alinhamento à tangente (align)

Quando align="True", o objeto orienta-se de acordo com a direção de movimento na curva. Após o glTranslatef com a posição calculada, é aplicada uma rotação adicional através de glMultMatrixf com uma matriz construída por Gram-Schmidt.

```

void buildAlignMatrix(float* deriv, float* matrix) {
    // Normalizar tangente → eixo X
    // Gram-Schmidt: Y = Ztmp × X, normalizar
    // Z = X × Y
    // Preencher matrix[16] coluna-major
}

```

4. Engine – VBOs com Índices (EBO)

A renderização foi migrada do modo imediato (glBegin/glEnd) para VBOs com EBO (Element Buffer Object). Esta abordagem combina eficiência de GPU com economia de memória, e foi projetada para facilitar a extensão na Fase 4.

4.1 Dois buffers na GPU

Cada modelo carregado cria dois buffers na GPU:

- VBO (GL_ARRAY_BUFFER): contém os vértices únicos após deduplicação, em formato flat [x0,y0,z0, x1,y1,z1, ...]
- EBO (GL_ELEMENT_ARRAY_BUFFER): contém os índices das faces em formato flat [v1,v2,v3, ...], do tipo unsigned int

A deduplicação de vértices foi preservada da Fase 2 — o mesmo mapa de chave string para índice garante que cada posição única é guardada apenas uma vez.

4.2 Upload e desenho

```

// Upload VBO
glGenBuffers(1, &modelData.vboId);
glBindBuffer(GL_ARRAY_BUFFER, modelData.vboId);
glBufferData(GL_ARRAY_BUFFER, size, flatVerts.data(), GL_STATIC_DRAW);

// Upload EBO
glGenBuffers(1, &modelData.eboId);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, modelData.eboId);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, size, flatIdx.data(), GL_STATIC_DRAW);

// Desenhando
glBindBuffer(GL_ARRAY_BUFFER, md.vboId);
glVertexAttribPointer(3, GL_FLOAT, sizeof(float)*3, 0);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, md.eboId);
glDrawElements(GL_TRIANGLES, md.indexCount, GL_UNSIGNED_INT, 0);

```

4.3 Preparação para a Fase 4

A estrutura foi deliberadamente preparada para a Fase 4. A struct Vertex contém apenas posição por agora, mas basta acrescentar os campos de normal e coordenadas de textura:

```
// Fase 3 (atual)           // Fase 4 (basta acrescentar)
struct Vertex {             struct Vertex {
    float x, y, z;           float x, y, z;
};                           float nx, ny, nz;
                             float u, v;
                             };
```

Os buffers são criados após `glutCreateWindow` porque só então existe um contexto OpenGL ativo — ao contrário da Fase 2, onde os modelos eram carregados antes do GLUT. O EBO não precisará de ser alterado na Fase 4, pois os índices são independentes dos dados de cada vértice.

5. Parser XML – Extensões da Fase 3

O parser foi estendido para detetar e processar as novas formas dos elementos `translate` e `rotate`. A distinção entre transformação estática e animada é feita pela presença do atributo `time`.

5.1 Novos tipos de transformação

Tipo	XML	Descrição
TRANSLATE	<translate x y z>	Translação estática (Fase 2)
TRANSLATE_ANIM	<translate time align>	Percurso sobre curva Catmull-Rom
ROTATE	<rotate angle x y z>	Rotação estática (Fase 2)
ROTATE_ANIM	<rotate time x y z>	Rotação contínua temporizada
SCALE	<scale x y z>	Escala estática (Fase 2)

6. Cena de Demonstração – Sistema Solar Dinâmico

A cena `solar_system_fase3.xml` estende a cena estática da Fase 2 com animações temporais em todos os corpos celestes e um cometa com trajetória Catmull-Rom.

6.1 Princípio geral de escalagem

Numa visualização do sistema solar é impossível respeitar simultaneamente as escalas reais de tamanho e de distância. Por isso foram usadas duas escalas independentes e intencionalmente exageradas (Escala de tamanho, Escala de distância, Escala de tempo).

6.2 Hierarquia de rotações

Corpo	Período orbital	Filhos	Período dos filhos
Sol	–	Mercúrio ... Netuno	10s – 1400s
Terra	40s	Lua	5s
Marte	80s	Fobos	3s
Júpiter	200s	Io, Europa	4s, 7s
Saturno	500s	Anel	300s (próprio)

6.3 Tamanho dos planetas – justificação

Corpo	Scale	Raio real (km)	Justificação
Sol	4.0	696.000	Referência visual central da cena
Mercúrio	0.3	2.440	O mais pequeno; visível mas menor que Vénus
Vénus	0.6	6.051	Semelhante à Terra, ligeiramente menor
Terra	0.7	6.371	Referência de tamanho dos planetas interiores
Lua	0.25	1.737	~35% da Terra — exagerado para visibilidade
Marte	0.5	3.390	Menor que Terra mas maior que Mercúrio
Fobos	0.15	11	Muito pequeno; exagerado para ser visível
Júpiter	1.5	69.911	O maior planeta; maior que todos os outros
Io / Europa	0.20/0.18	1.821/1.561	Exageradas para visibilidade; io ligeiramente maior
Saturno	1.2	58.232	Ligeiramente menor que Júpiter, como na realidade
Anel Saturno	3.5x0.05x3.5	--	Plano XZ achatado em Y para simular anel; raio ~3x o planeta
Urano	0.9	25.362	Gigante de gelo; entre Saturno e Neptuno em tamanho visual
Neptuno	0.85	24.622	Muito semelhante a Urano, ligeiramente menor
Cometa	0.5	--	Mantém o teapot reconhecível sem dominar a cena

6.4 Distâncias orbitais – justificação

Planeta	Distância usada	Distância real (UA)	Justificação
Mercúrio	8	0.39	Folga suficiente para não colidir com o Sol
Vénus	12	0.72	Proporção $\sim 1.5\times$ Mercúrio, coerente com $0.72/0.39 \approx 1.85$ reais
Terra	16	1.00	Referência de distância; $\sim 2\times$ Mercúrio
Marte	22	1.52	$\sim 1.4\times$ Terra, próximo do real ($1.52\times$)
Júpiter	32	5.20	Salto maior para separar cinturão de asteroides dos gigantes
Saturno	45	9.58	$\sim 1.4\times$ Júpiter; separação suficiente para o anel ser visível
Urano	58	19.2	Progressão uniforme para os gigantes de gelo
Neptuno	68	30.1	Limite da cena; confortavelmente dentro do frustum (far=1000)

6.5 Períodos orbitais – justificação

Os períodos foram definidos usando a Terra como referência temporal com 40 segundos. Os planetas interiores seguem proporções muito próximas dos períodos reais relativos à Terra.

Planeta	Tempo (s)	Período real (dias)	Fator real vs Terra	Fator usado vs Terra (40s)
Mercúrio	10	88	$0.24\times$	$10/40 = 0.25\times \approx$ real
Vénus	25	225	$0.62\times$	$25/40 = 0.63\times \approx$ real
Terra	40	365	$1\times$	Referência
Lua	5	27	$0.074\times$	$5/40 = 0.125\times$ — ligeiramente rápida para ser visível
Marte	80	687	$1.88\times$	$80/40 = 2.0\times \approx$ real
Fobos	3	0.32	muito rápido	Exagerado para ser visível na demo
Júpiter	200	4333	$11.9\times$	$200/40 = 5\times$ — comprimido para observabilidade
Io	4	1.77	—	Rápido e visível; mais rápido que Europa
Europa	7	3.55	—	Proporção Io:Europa $\approx 1:2$, próxima da real
Saturno	500	10759	$29.5\times$	$500/40 = 12.5\times$ — comprimido para observabilidade
Anel Saturno	300	—	—	Rotação própria mais rápida que a orbital de Saturno
Urano	900	30687	$84\times$	$900/40 = 22.5\times$ — movimento ainda perceptível

Netuno	1400	60190	165×	1400/40 = 35× — o mais lento, quase estático na demo
--------	------	-------	------	--

7. Cometa – Curva Catmull-Rom

7.1 Número de pontos de controlo

A curva do cometa usa 12 pontos de controlo. Esta escolha garante a suavidade da elipse e a periodicidade (repetindo os primeiros 3 pontos no final) para um ciclo contínuo. Com menos de 8 pontos a órbita elíptica ficaria com cantos perceptíveis nos quadrantes. Com 16 pontos não havia melhoria visual perceptível face a 12. Para que a curva feche suavemente sem descontinuidade, os primeiros 3 pontos são repetidos no final (os pontos 0, 1 e 2 repetem-se nas posições 8, 9 e 10), garantindo continuidade C^1 na junção do ciclo. O cometa completa a órbita em 30 segundos — rápido o suficiente para ser claramente observável numa demo, mas não tão rápido que o alinhamento tangencial seja difícil de perceber. O parâmetro `align="True"` foi ativado porque o modelo do teapot tem orientação não-simétrica: sem alinhamento o teapot giraria aleatoriamente ao longo da órbita; com `align="True"` o modelo aponta sempre na direção do movimento.

8. Compilação e Execução

```
# Compilar Generator
g++ generator.cpp -o generator

# Compilar Engine
g++ engine.cpp camera.cpp parser.cpp tinyxml2.cpp -o engine -lglut -lGL -lGLU

# Gerar modelos
./generator sphere 1 20 20 sphere.3d
./generator plane 2 10 plane.3d
./generator bezier teapot.patch 10 comet.3d

# Executar
./engine solar_system_fase3.xml
```

9. Decisões de Implementação

O uso de EBO em vez de array flat foi motivado pela eficiência e facilidade de transição para a Fase 4. A função Catmull-Rom calcula posição e derivada em simultâneo para otimização. O referencial de alinhamento é construído por Gram-Schmidt: a tangente normalizada define o eixo X; o produto externo com (0,1,0) dá o eixo Y (com fallback para (0,0,1) se a tangente for paralela a Y); e $Z = X \times Y$. Este método é numericamente estável e produz sempre um referencial ortonormal válido. Os VBOs e EBOs são criados após `glutCreateWindow` porque só então existe um contexto OpenGL ativo — ao contrário da Fase 2 onde os modelos eram carregados antes do GLUT.

10. Conclusão

A Fase 3 consolida o motor 3D com três contribuições técnicas relevantes. A integração de patches de Bézier bicúbicos no gerador permite criar geometria livre e orgânica, ultrapassando as limitações das primitivas geométricas simples das fases anteriores. As animações temporais — rotações contínuas e translações sobre curvas Catmull-Rom com alinhamento tangencial — conferem dinamismo à cena sem comprometer a estrutura hierárquica de grupos já existente. Por fim, a migração para VBOs com EBO representa uma mudança arquitetural importante: a renderização deixa de depender do modo imediato e passa a tirar partido da GPU de forma eficiente, reduzindo a largura de banda CPU-GPU através da deduplicação de vértices.

A cena de demonstração valida as três componentes em conjunto: os planetas orbitam com períodos proporcionais aos reais, as luas herdam as posições dos planetas pela hierarquia de grupos, e o cometa percorre a curva Catmull-Rom alinhando-se à direção de movimento. As decisões de implementação — nomeadamente o uso de EBO em detrimento de um array flat e o cálculo simultâneo de posição e tangente na curva — foram tomadas com vista à Fase 4, onde a adição de normais, coordenadas de textura e iluminação será uma extensão incremental sobre a base aqui construída.